

Netflix Prize Dataset

Recommendation System

Building an Intelligent Movie Recommendation Engine

A Presentation by – Parth Goyal & Aarav Singh

Problem Overview

The Challenge

- Streaming platforms host tens of thousands of movies
- Users face choice paralysis
- How to accurately connect the right user with the right movie?

Our Objective

- Build intelligent recommendation engine
- Predict user preferences accurately
- Generate personalized Top-K recommendations

Exploratory Data Analysis

Rating Bias

Ratings lean heavily positive (4-5 stars dominate)

Long Tail Distribution

Few blockbusters, thousands of niche movies with minimal ratings

Sparsity Challenge

User-item matrix is 98% empty.
Naive approaches won't work.

Our Approach

Collaborative Filtering Techniques

Memory-Based

User-Based KNN

"Users who agreed in the past will agree in the future."

- Finds nearest neighbors based on rating history

Model-Based

Matrix Factorization (SVD)

- Decomposes massive sparse matrix into lower-dimensional latent factors
- Represents hidden features (genre affinity, director preference, etc.)

System Architecture

From Raw Data to Recommendations

1

Data Pipeline

Custom parsing of Netflix format into structured triplets

2

Model Training

scikit-surprise: SVD and KNN model training

3

Evaluation Engine

Custom RMSE and MAP@10 metrics

4

Interactive Dashboard

Streamlit app for visualization & recommendations

Experimental Results

Matrix Factorization (SVD)

RMSE

0.9736

MAP@10

0.7102

✓ Strong performance on evaluation metrics

User-Based KNN

Scalability Challenge

- Failed to scale to full dataset
- Out of Memory (OOM) constraints
- Proves necessity of dimensionality reduction for production systems

Recommendation Quality

Moving Beyond Accuracy

The system generates meaningful, ranked lists—not just predicts ratings.

Real Example:

User Profile: Highly rated **The Matrix** and **Inception**

System Recommends:

Blade Runner

Interstellar

✓ Latent factors effectively capture implicit genre and thematic relationships

Interactive Dashboard

Bringing Data to Life

Streamlit Web Application

- Select any user from the database
- View their historical favorite movies
- Instantly generate personalized Top-10 recommendations
- Explore visual EDA insights

Key Insights & Future Work

What We Learned

- Matrix Factorization solves the sparsity problem
- Outperforms memory-based methods at scale
- "Cold Start" problem remains for new users/movies

What's Next

- Build Hybrid Model incorporating metadata
- Experiment with Neural Collaborative Filtering
- Deploy as scalable API via Docker & AWS

Thank You
